RegisterFile.scala 逐行讲解

本文对应同步 1RW 改造后的 RegisterFile.scala。空行不单独解释，跨行语句及其结束括号合并说明。此前生成的同名 PDF 对应旧的异步读版本，尚未更新，当前以本文和源码为准。

先理解数据怎样存放

VRF 是向量计算单元旁边的寄存器存储。指令从这里读取操作数，计算结果和 load 数据也写回这里。

默认配置 VLEN=4096、DLEN=256，有 32 个架构向量寄存器。每个寄存器拆成 16 块，每块称为一个 element group，编号简称 `eg`。全部寄存器共有 512 块，总容量为 16 KiB。

每块 256 位数据再拆成 4 个 64 位 lane 切片。每个 lane 有 8 个 bank，每个 bank 保存 64 行。总共 32 个 64 行乘 64 位的同步单端口 SRAM 实例。它们按 8 个 bank 编号共同调度，不是 32 个任意地址的 256 位访问端口。

例如读取 `v1` 的第 3 块，编号都从 0 开始：

```
eg   = 1 * 16 + 3 = 19
bank = 19 % 8 = 3
row  = 19 / 8 的整数部分 = 2

lane 0 的 bank 3 第 2 行 -> bit  63:0
lane 1 的 bank 3 第 2 行 -> bit 127:64
lane 2 的 bank 3 第 2 行 -> bit 191:128
lane 3 的 bank 3 第 2 行 -> bit 255:192
```

4 个切片同时访问，返回后重新拼成一块 256 位数据。

读源码前需要的词和语法

写法或术语	通俗解释
`val`	给对象取名字；是否保存状态，要看右侧是不是 `Reg`。
`UInt(64.W)`	64 位无符号硬件信号。
`Module(new X)`	创建一份硬件模块。
`IO(new Bundle {...})`	定义模块的一组输入输出。
`Flipped`	将接口的输入输出方向翻转，以接到接口另一端。
`:=`	连线，本身不增加一拍延迟。
`<>`	按接口方向批量连线。
`Reg`、`RegInit`	在时钟边沿保存状态；后者有复位值。
`RegEnable(data, en)`	`en` 成立时，在时钟边沿保存 `data`。
`RegNext(x, false.B)`	下一拍得到这一拍的 `x`，复位后为假。

写法或术语	通俗解释
`Wire`、`WireInit`	当前拍的组合信号；后者提供默认驱动。
`when`	描述条件电路，内部对寄存器的更新在时钟边沿生效。
`for`、`map`、`Seq.tabulate`	用 Scala 生成多份电路或连接，不是 CPU 逐个执行访问。
`Vec`、`MixedVec`	一组同形状信号、一组允许不同形状的信号。
`Decoupled`	含 `valid`、`ready`、`bits`；二者同时为真称为 `fire`。
`Valid`	只有 `valid` 和 `bits`，接收方不能反压，必须及时接收。
`Mux(c,a,b)`	`c` 为真选 `a`，否则选 `b`。
`Mux1H`	由独热选择码选一路，底层是与后按位或，不会替你检查选择码。
`UIntToOH(3)`	编号转独热码，如 8 位的 `00001000`。
`===`、`&&`、`!`	硬件比较相等、布尔与、布尔取反。
`&`、`|`、`~`	按位与、或、取反。
`x(hi,lo)`	取某一段位。
`Cat`	将多个位串拼起来。
`oldest`	上游提供的优先标记，这里不重新比较全局指令年龄。
pipeline 写	到达时没有 `ready`，必须事先预约未来的写时刻。
LL 写	可通过 `valid/ready` 等待接收的写回，包含初始化、load、迭代运算等来源。
写掩码	每个数据位有一个使能位，1 表示更新，0 表示保持。
mask 读	读取向量指令的掩码数据，与写使能不是一回事。

两种读握手

内部 `SramReadIO` 将“接受请求”和“返回数据”分开：本拍 `req.fire`，下一拍 `resp.valid`。返回没有 `ready`，接收方必须在有效拍取走数据。

外部 `VectorReadIO` 保留 Saturn 原接口，调用方在 `req.fire` 同拍读取 `resp`。包装层先替调用方发起 SRAM 读，直到返回数据匹配当前请求才给调用方 `ready`。因此外部 `fire` 表示数据已经到手，内部 `fire` 表示存储刚接到读地址。

第 1-39 行：导入、内部读接口和兼容收集层

行号	解释
1	指定这些类所属的 Scala 包。
3-4	导入 Chisel 基础类型、握手、仲裁器、选择器等工具。
5	导入参数系统，用隐式参数传递向量长度等配置。

行号	解释
6-7	导入 Rocket 的基础模块、接口和工具。
8	导入 EVU 的公共接口及向量参数。
9	导入仓库已有的同步 SRAM 封装。
11	说明内部读响应固定延迟一拍，没有返回反压。
12	定义读接口，继承基础 Bundle 并取得向量参数。
13	读请求带地址和 `oldest`，允许存储端用 `ready` 让请求等待。
14	读响应带有效位和 `dLen` 位数据，方向为存储返回到调用方。
15	结束接口定义。
17-18	定义兼容收集层，保持已有调用方的取数方式。
19-22	左边接旧接口客户端，右边接新的同步读接口。
24	`pending` 记录是否已有在途请求，复位后没有。
25	保存发出去的 `eg`，用于确认返回仍属于当前地址。
26	客户端有请求、没有在途读且不在复位时，才向存储发新请求。
27	地址和优先标记直接送给存储。
28-29	只有在途响应有效、客户端仍有请求且地址匹配，才向客户端报告可以取数。
30	返回数据直接送给客户端；客户端只可在外部 `fire` 时消费它。
32-34	存储接收请求时，设置在途标志并保存地址。
35-37	收到响应便结束这次在途读。客户端撤回请求或改了地址时，不确认给客户端，而是丢弃旧响应。
38-39	结束条件和模块。

每个逻辑读端只有一个在途读。等待响应期间不发第二个读，持续请求时最多每两拍完成一次；这是兼容收集层的吞吐限制，并非物理 SRAM 每两拍才能访问。

第 41-69 行：读仲裁器

行号	解释
41-42	创建 `n` 个输入、一个输出的读请求仲裁器。
44	在生成电路前检查输入数量大于零。
45	创建一个轮询仲裁器，负责普通请求的公平轮转。
46-49	将所有请求的有效位、内容接到轮询仲裁器。
51	找出有效且带 `oldest` 标记的输入。
52	如果有多个这样的输入，选编号最小的一个，生成独热选择码。

行号	解释
53	判断是否需要覆盖轮询结果。
55-58	有优先请求时输出它，否则输出轮询选择。
59-61	同样输出最终获准者的编号。
62	只有采用轮询结果且下游接收时，才推进轮询仲裁器。
64-67	将最终接收条件传回各输入。优先模式只允许一个获准输入握手。
68-69	结束循环和模块。

第 71-97 行：按 bank 分发读地址并延迟路由响应

行号	解释
71-75	定义 `n` 个逻辑读输入、`banks` 个物理 bank 方向的输出，均使用同步读协议。
77	每个 bank 实例化一个仲裁器。
78-80	将各仲裁器输出接到对应的 bank 输出。
82	计算选择 bank 需要多少地址位，8 个 bank 需要 3 位。
84-85	对每个输入，取 `eg` 低位并转成 bank 独热选择码。
86-87	只将请求有效位送到选中的 bank 仲裁器。
88	去掉 bank 位，向后传 bank 内行号。字段仍叫 `eg`，此处含义已变为行号。
89	保留 `oldest` 标记。
90-91	结束分发，并把被选中 bank 的接收条件传回输入。
92	请求成交时保存本次 bank，不能在下一拍用新请求的 bank 选旧响应。
93	将本拍是否读成交延迟一拍。
94	上拍确有成交，且记录的 bank 有响应时，才对这个输入返回有效数据。
95	按保存的 bank 选择返回数据。
96-97	结束循环和模块。

例如本拍请求 bank 3，下一拍输入已改成 bank 4，返回的旧数据仍必须来自 bank 3。第 92 行的寄存器就是为了记录这件事。

第 99-152 行：外部接口、包装层和物理接口

行号	解释
99-106	定义对外接口，并接收各类读写端口数量及最大写回深度。
107	数据读按组组织，各组端口数可以不同，使用旧的同拍取数接口。

行号	解释
108	掩码读也按组组织。
109	流水线写预约接口，负责未来 bank 写时刻的可用性。
111	流水线实际写回只有有效位和数据，没有 `ready`。
112	LL 写回有 `ready`，允许等待。
113	仅在调试配置下输出所有 `eg` 的快照。
114	接口定义结束。
116-117	定义包装模块。`sramSuffix` 是 SRAM 名称后缀，不改变功能；同 JVM 多次测试展开可使用不同名称避开封装的全局定义缓存。
119-120	创建外部 IO 和真正保存数据的 `AraRegisterFile`。
121-124	写预约、两类写回和可选调试输出直接连接。
126-130	定义生成读收集层的方法，每次调用都创建一份硬件实例。
131-133	遍历所有数据读端，每端接一个收集层。
134-136	对所有掩码读端做相同处理。
137	包装模块结束。
139-145	定义内部物理 VRF 接口的参数。
146-147	数据读和掩码读使用带响应有效位的同步接口。
148-151	写预约、两类写回和调试接口与外部含义相同。
152	内部接口定义结束。

第 154-204 行：建立物理存储并接好读组

行号	解释
154-162	定义实际 VRF 模块，每个 `eg` 横跨多个 64 位 lane，每个 lane 8 个 bank。
164	每份切片的宽度固定为 64 位。
165	`dLen / 64` 算出 lane 数，默认 4。

行号	解释
166-167	bank 数固定为 8，bank 地址位数为 3。
168	总 `eg` 数除以 8，得到每个 bank 的行数。
169	算出行地址位宽，并确保至少一位。
171	检查 `dLen` 能整分为 64 位切片。
172	检查 `eg` 总数能均分到 8 个 bank。
173-174	检查逻辑写预约的 bank 数与实际存储一致。
176	创建带同步读响应的内部接口。
178	对每个 lane 和 bank 创建一份存储。
179-180	使用每行 64 位的 `SRAMTemplate`，启用单端口和逐位写使能，设置模块名称。默认读延迟为一拍。
181	结束存储数组生成。
183-186	调试时额外生成一份寄存器影子副本，用来观察内容，不给真实 SRAM 增加读端口。
188	完整 `eg` 低 3 位选 bank。
189	完整 `eg` 去掉低 3 位得到行号。
191-192	每组数据读和掩码读分别建立一个第一级 crossbar。
194-199	将模块的读输入连接到各组 crossbar。
201-203	收集所有 crossbar 的 bank 输出，数据读和掩码读共享第二级 bank 仲裁。
204	要求至少有一个读请求组。

真实 SRAM 不由这里的 `RegInit` 清零。初始化写入由上层 `RegisterAccess` 逐个地址发起；调试影子复位为零不代表 SRAM 内容已经清零。

第 206-242 行：识别流水线写回，缓冲 LL 写回

行号	解释
206-209	按 bank 检查哪些流水线写回有效且目标是本 bank。
210	每个 bank 是否有流水线写回到达。
211-213	根据匹配结果选出写数据、逐位掩码和完整 `eg` 地址。
215-217	每个 bank 另建一个 LL 写仲裁器，从多个来源中选一个。

行号	解释
218-220	每个 bank 建一项 LL 缓冲及有效位，切断 load mask 读与写 ready 的组合依赖。
221-223	先为每个 bank、每个 LL 来源准备 ready 连线。
224-228	将当前来源送入目标 bank；流水线写到达时，不接收同 bank 的新 LL 写。
229-230	将目标 bank 的接收条件送回对应 LL 来源。
232	缓冲为空可以接收；没有流水线写时，旧 LL 写会提交，也可以接替一个新 LL 写。
233-235	新 LL 写被接收时，把地址、数据、掩码保存到缓冲并置有效。
236-237	没有新写接替、旧 LL 写获得提交机会时，清空缓冲。
238-239	结束条件和 bank 循环。
240-242	将各 bank 的 ready 汇总，反馈给每个 LL 来源。地址只会选中一个 bank。

LL `fire` 表示进入缓冲，不表示已经写入 SRAM。旧 LL 写本拍提交、新 LL 写本拍进入缓冲，二者访问不同层次，可以同时发生。

第 244-267 行：提前预约未来的流水线写回

行号	解释
244-245	建立各流水线写来源的阻塞信号，默认不阻塞。
246-248	对外报告能否预约，为阻塞信号取反。
249-250	每个物理 bank 建一个未来时刻调度器，优先类和普通类各留一组槽。
251-255	优先类请求需满足“有预约请求、选择这个 bank、带 oldest”，并传入结果到达深度和实际发射信号。
256-258	这个 bank 无法提供目标时刻时，将该来源标记为阻塞。
260-262	对没有 oldest 标记的普通类请求，连接另一组调度器输入。
263-265	普通类预约失败也要阻塞对应来源。
266-267	结束来源循环和 bank 循环。

目的在于避免计算结果已经到达，却发现写端口没有空位。实际写回没有反压，所以不能等到最后再决定让它等待。

第 269-297 行：选择 bank 读地址并合并相同行

行号	解释
269-272	每个 bank 建第二级仲裁器，输入是各第一级 crossbar 对这个 bank 的候选。
274-276	逐个候选检查相同行的合并条件。
277-280	`sameEarlier` 表示更小编号处已经有相同行的有效请求。
281-285	`sameOldestEarlier` 表示更小编号处已有相同行的优先请求。
286-289	`anyOldestInGroup` 表示整个相同行请求集合中是否有人带优先标记。
290-292	相同行只选一个代表：有优先请求时选最小编号的优先请求，否则选最小编号请求。
293-294	只有代表参加第二级仲裁，但仍传入原地址和优先标记。
296	只有物理读真正成交且地址相等时，才给候选 ready。相同行的其他候选也能共享此次访问，不同行不能误收。
297	结束候选循环。

这里合并的是各读组已经选出的候选。某组内部尚未通过第一级仲裁的另一个请求，不能同时参与第二级合并。

第 299-345 行：单端口访问、返回路由和写入

行号	解释
299	本 bank 是否有流水线写。
300	没有流水线写时，已缓冲的 LL 写才有提交机会。
301	两者任一存在，本拍就用于写入。
302	优先选流水线写地址，否则选 LL 缓冲地址。
303-304	有写或复位时，不允许接受新的读请求，落实每 bank 每拍读写二选一。
306-309	拼出本拍写地址、数据和逐位掩码。
311	将各 lane 的 SRAM 返回数据按高低位顺序拼成完整 `dLen` 位。
312-315	每个请求组将本拍成交位延迟一拍，作为返回有效位；返回数据为同一 bank 的 SRAM 结果。只有上拍获准的接收者得到有效响应。
316	结束返回连接。

行号	解释
318	不在复位且存在写时，才真正启用 SRAM 写口。
320-323	如果同 bank 同时有多个流水线写回，触发断言。断言负责发现违规，不会自动排队解决冲突。
324-326	遍历 lane，确定它在完整数据中的起始位，并找到对应 SRAM。
327	将物理读成交信号和行号送入该 lane 的同步读口。
328-329	将写使能、64 位数据切片、行号、唯一 way 使能和 64 位位掩码送到写口。
330-331	断言 SRAM 能接受已发出的访问，避免封装端口停顿时静默丢失请求。
332	结束 lane 循环。
334-338	真实写提交时，也更新可选调试影子。影子用寄存器按位合并，不是对 SRAM 做第二次读。
339-340	结束调试更新和 bank 循环。
342-344	开启调试时，将影子寄存器逐项接到输出快照。
345	结束整个 VRF 模块。

逐位写使能直接决定哪些 SRAM 位更新，不再为部分写先读旧值。比如 mask 为 `...00000100` 时，只能更新 bit 2，其他位必须保持。仅支持字节使能的 SRAM 宏不能直接满足这个接口。

把几拍的行为串起来

假设没有写冲突，客户端持续请求 `eg=19`：

周期	行为
N	收集层向内部发请求，bank 3 接受 row 2；保存请求地址和返回接收者。外部 ready 仍为 0。
N+1	SRAM 返回 4 个 lane 的数据，内部 resp.valid 为 1；收集层检查客户端仍请求 eg 19，然后拉高外部 ready。客户端在本拍结束边沿取数。
N+2	客户端可以发下一次请求，或者已由自己的操作数寄存器保存上一份结果。

假设 N+1 有同 bank 写入，它不能再接受新的读地址，但可以同时返回 N 拍读取的数据。这两件事分别使用不同周期的端口。

一次 LL 写的过程则是先握手进入 bank 缓冲，下次没有流水线写抢占时提交到 SRAM。流水线写优先于缓冲 LL 写，缓冲 LL 写优先于新读。

与其他文件的关系

- `common/Bundles.scala` 定义原有的 `VectorReadIO`、读请求和逐位写数据结构。
- `backend/RegisterAccess.scala` 连接所有 VRF 客户端，发初始化清零写，初始化完成前禁止读请求进入收集层。
- `backend/Backend.scala` 对前端 index/mask 读施加依赖门控，防止提前采到旧值。
- `backend/LoadSequencer.scala` 对 load mask 读施加依赖门控。
- `backend/ExecuteSequencer.scala`、`StoreSequencer.scala` 等仍在外部 `req.fire` 时保存数据。
- `EvuVectorBackend.scala` 已将 Saturn 子系统放在复位或 flush 的共同边界内；同步读在途状态和返回路由寄存器也随之清除。
- `utility/src/main/scala/utility/sram/SRAMTemplate.scala` 封装同步存储；`SramProto.scala` 的 `SramArray1P` 用 `SyncReadMem.readWrite` 描述同一个物理读写端口。

这次改变的是 RTL 存储模型、返回协议及调用方时序适配。实际工艺硬宏仍需满足一拍读和逐位写使能，并完成宏映射、综合与时序验证。